

Aula 14 - Teste de Integração

Descrição: Analisando os testes de integração em PHP utilizando o **PHPUnit** exige compreendermos que, diferentemente dos testes unitários que testam classes ou métodos isolados, os testes de integração garantem que diferentes partes do seu sistema — ou serviços externos — funcionem perfeitamente em conjunto. Nesta aula vamos estruturar, executar e aplicar boas práticas para testes envolvendo **Bancos de Dados e APIs Externas (como o ViaCEP)**.

1. Testes de Integração com Banco de Dados

Testar a integração com o banco de dados significa verificar se as suas *queries*, *models* ou repositórios estão gravando, lendo, atualizando e deletando os dados corretamente na base real.

Desafios

- **Estado dos Dados:** Um teste não pode interferir no outro. Se o Teste A insere um usuário, o Teste B não deve falhar por encontrar um usuário inesperado.
- **Lentidão:** Testes de banco são naturalmente mais lentos que testes unitários devido ao I/O de disco e rede.

Soluções e Boas Práticas

1. **Banco de Dados Exclusivo para Testes:** Nunca rode testes no banco de produção ou desenvolvimento local. Crie um banco específico (ex: meu_projeto_testing). O uso de **SQLite em memória** (:memory:) é muito comum para acelerar a execução, embora usar o mesmo motor de produção (MySQL/PostgreSQL) via Docker garanta maior fidelidade.
2. **Transactions (Transações):** A melhor forma de manter o banco limpo é iniciar uma transação no método setUp() do PHPUnit e fazer um rollBack() no método tearDown(). Assim, nada é de fato persistido no disco ao final do teste.
3. **Seeders e Factories:** Utilize geradores de dados para popular o banco com o estado necessário antes da asserção (ex.: generatedata.com).

Exemplo:

Como destacamos, o maior desafio aqui é o estado dos dados (garantir que um teste não interfira no outro). A solução ideal é usar Transactions (Transações), iniciando a transação no setUp() e desfazendo tudo com rollBack() no tearDown().

O Código (UserRepositoryTest.php)

PHP

```
<?php
```

```
use PHPUnit\Framework\TestCase;
```

```
class UserRepositoryTest extends TestCase
```

```
{  
    private PDO $db;  
    private UserRepository $repository;
```

```
// Executado ANTES de cada teste
```

```

protected function setUp(): void
{
    // Conecta em um banco exclusivo de testes (ex: SQLite em memória para rapidez)
    $this->db = new PDO('sqlite::memory:');
    $this->db->exec("CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, email TEXT)");

    // Inicia a transação para que nada seja salvo definitivamente no disco
    $this->db->beginTransaction();

    $this->repository = new UserRepository($this->db);
}

// O teste em si
public function test_deve_salvar_um_usuario_no_banco_de_dados(): void
{
    // 1. Dados de teste (simulando um Factory/Seeder)
    $userData = ['name' => 'João Silva', 'email' => 'joao@email.com'];

    // 2. Executa a ação
    $this->repository->save($userData);

    // 3. Asserção: Verifica se o dado realmente foi gravado corretamente
    $savedUser = $this->repository->findByEmail('joao@email.com');

    $this->assertNotNull($savedUser);
    $this->assertEquals('João Silva', $savedUser['name']);
}

// Executado DEPOIS de cada teste
protected function tearDown(): void
{
    // Desfaz tudo o que foi feito no teste, deixando o banco limpo para o próximo
    $this->db->rollBack();
}
}
?>

```

Explicação

- **setUp()**: Antes de o teste rodar, nós preparamos o ambiente. Criamos a tabela e abrimos uma transação (beginTransaction). É como se abrísssemos um rascunho de alterações no banco.
 - **O Teste**: Inserimos o "João Silva" e perguntamos para o banco se ele realmente está lá. Se o banco responder que sim, o teste passa.
 - **tearDown()**: Assim que o teste termina, o rollBack() entra em ação. Ele "deleta" o rascunho. Dessa forma, se houver um "Teste B", o banco estará totalmente limpo e o João Silva não estará lá para atrapalhar.
-

2. Testes de Integração com APIs Externas (Ex: ViaCEP)

Quando seu sistema depende de uma API externa como o **ViaCEP** (para buscar endereços a partir do CEP), você enfrenta um dilema: se você fizer requisições reais para a API durante os testes, sua suíte ficará **lenta**, sujeita a **falhas de rede** e pode ser bloqueada por **Rate Limiting** (limite de requisições).

A Abordagem Correta: Mocking do Cliente HTTP

Em vez de bater na API real do ViaCEP a cada execução, nós "mockamos" (simulamos) as respostas HTTP. Isso garante que o seu código que *processa* a resposta seja testado, sem depender da internet. Se você estiver usando o **Guzzle** (o cliente HTTP mais popular no PHP), ele possui uma ferramenta nativa excelente para isso: o MockHandler.

Nota sobre Contratos: Para garantir que a API real não mudou seu formato de resposta sem você saber (quebrando seu mock), é recomendável criar testes de *Contrato* (ou testes End-to-End agendados) que rodam periodicamente contra a API real, mas separá-los da suíte de testes principal executada no seu CI/CD diário.

Exemplo

Bater na API real do ViaCEP a cada teste gera lentidão e falhas de rede. A abordagem correta é usar o MockHandler do Guzzle para fingir que fomos até a internet, testando apenas como o nosso código processa essa resposta.

O Código (ViaCepServiceTest.php)

PHP

```
<?php
use PHPUnit\Framework\TestCase;
use GuzzleHttp\Client;
use GuzzleHttp\Handler\MockHandler;
use GuzzleHttp\HandlerStack;
use GuzzleHttp\Psr7\Response;

class ViaCepServiceTest extends TestCase
{
    public function test_deve_retornar_o_endereco_correto_ao_buscar_por_cep(): void
    {
        // 1. Criamos a resposta "falsa" que simula o formato real do ViaCEP
        $mockResponseBody = json_encode([
            "cep" => "01001-000",
            "logradouro" => "Praça da Sé",
            "bairro" => "Sé",
            "localidade" => "São Paulo",
            "uf" => "SP"
        ]);

        // 2. Injetamos essa resposta no MockHandler do Guzzle
        $mockHandler = new MockHandler([
```

```

        new Response(200, [], $mockResponseBody)
    ]);

    $handlerStack = HandlerStack::create($mockHandler);
    $client = new Client(['handler' => $handlerStack]);

    // 3. Passamos o cliente "mockado" para o nosso serviço
    $viaCepService = new ViaCepService($client);
    $endereco = $viaCepService->buscar('01001000');

    // 4. Asserção: Garantimos que o nosso código processou o JSON corretamente
    $this->assertEquals('Praça da Sé', $endereco->getLogradouro());
    $this->assertEquals('São Paulo', $endereco->getCidade());
}
}
?>

```

Explicação

- **O Mock (MockHandler):** Nós criamos um JSON idêntico ao que o ViaCEP nos devolveria se estivéssemos conectados à internet.
- **O Truque:** Configuramos o cliente HTTP (Guzzle) para que, em vez de disparar uma requisição para a web, ele simplesmente pegue essa resposta falsa que deixamos programada.
- **O Objetivo:** O objetivo aqui não é testar se o site do ViaCEP está no ar, mas sim testar se o nosso código sabe ler o JSON e transformá-lo em um objeto de endereço válido no nosso sistema. Isso deixa a suíte de testes extremamente veloz e imune a quedas de internet.

Conclusão: O Papel Estratégico dos Testes de Integração

Os testes de integração funcionam como a engrenagem que garante a confiabilidade do ecossistema de um software. Enquanto os testes unitários validam a lógica isolada de funções e métodos de forma cirúrgica, os testes de integração sobem um degrau na arquitetura para validar a **comunicação real entre componentes internos e serviços externos**. Eles respondem à pergunta crucial: *"Nosso código funciona bem quando entra em contato com o mundo exterior?"*

Fica claro que realizar testes de integração exige um equilíbrio inteligente entre **fidelidade** e **isolamento**, dividindo as estratégias em duas frentes principais:

1. O Equilíbrio no Armazenamento (Banco de Dados)

Para componentes locais persistentes, como bancos de dados, **a fidelidade é indispensável** para garantir que as queries, modelos e repositórios operem corretamente. No entanto, para mitigar os problemas de lentidão e o risco de um teste interferir no resultado do outro (estado dos dados), o uso de ambientes isolados — como **SQLite em memória** ou ambientes isolados via Docker — combinado com o controle rigoroso de **transações** (**`beginTransaction`** e **`rollback`**), prova-se a melhor abordagem técnica. Isso permite que os testes sejam rápidos, confiáveis e que o banco termine exatamente como começou: limpo.

2. O Isolamento de Dependências Instáveis (APIs Externas)

Quando o sistema depende de terceiros (como a API do ViaCEP), a abordagem muda drasticamente. Fazer

requisições reais para a internet sabota a estabilidade dos testes devido a falhas de rede, lentidão e bloqueios por limites de requisição (*rate limiting*). A conclusão técnica ideal é o **Mocking do Cliente HTTP** (utilizando ferramentas como o **MockHandler** do Guzzle). Essa estratégia simula as respostas da API na memória e foca o teste no que realmente importa: a capacidade do *seu* sistema de processar e interpretar esses dados corretamente. Para garantir que a estrutura da API externa não mudou à revelia, utilizam-se testes de contrato isolados da suíte principal.

Resumo Final

Os testes de integração bem estruturados removem o medo de implantar alterações em produção. Ao dominar o uso de **bancos de dados temporários** e a **simulação inteligente de APIs**, o desenvolvedor cria uma esteira de automação (CI/CD) resiliente, veloz e imune a fatores externos, assegurando a integridade e a continuidade do software a longo prazo.